Syscalls Process Primer

user mode、syscall、address space、context switch、`fork/exec/wait` を、x86-64 の最小 kernel / user 境界として整理する学習用ノートです。`elf-linker-loader-primer` の `execve` / startup と `rust-os-book` の kernel 側をつなぎ、`go-runtime-primer` の syscall / netpoll の理解にも戻れるように構成しています。

> **編集注記**
> この文書は Linux kernel や xv6 の逐語訳ではありません。x86-64 の特権遷移、syscall entry/exit、process image、`fork/exec/wait` の役割がつながる順に再構成した版です。完全な OS 実装手順書ではなく、「user/kernel 境界で何が保存され、何が切り替わるのか」を一本の線で追うことを優先しています。

目次

Contents

はじめに 2
- この教材の主題 2
- なぜ今これを学ぶのか 3
- 対象と範囲 3
- x86-64 の説明方針 4
- 参照する一次資料 4
- 各章の見取り図 4

rings、trap、user mode 5
- なぜ権限境界が必要なのか 5
- page table 上の user / kernel 境界 5
- trap は kernel entry の一般形 5
- kernel stack が必要な理由 5
- user mode へ「降りる」とは何か 6
- `iretq` と `syscall/sysret` 6
- TSS と stack 切り替え 6
- この章の出口 6

syscall path 6
- syscall は「特別な関数呼び出し」ではない 7
- entry で何を保存するか 7
- dispatch table 7
- `copyin/copyout` はなぜ必要か 7
- 返り値と失敗 8
- syscall return 8
- `write` と `exit` から始める理由 8
- この章の出口 8

process と address space 8
- process は何を持つか 9
- address space が process を分ける 9

user stack をどう作るか ... 9
exec は何をするか ... 10
kernel stack と trap frame の置き場所 ... 10
scheduler と process 状態 ... 10
この章の出口 ... 10
fork、exec、wait、context switch ... 10
fork は何を複製するか ... 11
copy-on-write は改善策であって本質ではない ... 11
exec は process を消さず image を替える ... 11
context switch は何を切り替えるか ... 11
scheduler の最小像 ... 12
exit と zombie ... 12
wait は何を待つか ... 12
この章の出口 ... 12
付録と次の一歩 ... 13
章ごとの復習ポイント ... 13
小さな演習課題 ... 13
よくあるつまずき ... 13
exec が child を作る syscall だと思ってしまう ... 13
syscall return と context switch を同じものだと思ってしまう ... 13
user pointer を kernel でそのまま触ってしまう ... 13
zombie を「ただのリーク」と思ってしまう ... 14
trap frame を「割り込み専用構造」と思ってしまう ... 14
用語小事典 ... 14
次に読むなら ... 14
参考資料 ... 14
おわりに ... 15

はじめに

この文書は、user mode と kernel mode の境界で何が起きているのかを整理するためのノートです。`elf-linker-loader-primer` では `execve`、loader、`_start` までを扱いました。しかしその先で、ユーザコードがどうやって kernel へ入り、どうやって戻り、どうやって新しい process を作り、どうやって終了を回収するのかは別の話として残っています。`rust-os-book` もヒープまでで止まっており、その次に自然に現れるのが task、syscall、user process です。本書の役割は、その空白を埋めることです。

低レイヤ学習では、syscall と process 管理が別々の話に見えやすいですが、実際には強くつながっています。syscall は user space が kernel に仕事を頼む入口であり、process はその user space 自体を管理する単位です。`fork`、`exec`、`wait`、context switch、user stack、trap frame は、すべて「どの実行主体が、どの address space で、どの権限で走っているか」という一つの問いへ戻ってきます。

この教材の主題

本書の主線は次の 4 点です。

1. なぜ user mode / kernel mode を分けるのか
2. syscall entry/exit で、CPU 状態と stack をどう扱うのか

3. process image と address space をどう作り替えるのか
4. `fork/exec/wait` と context switch がどうつながるのか

この順序を取る理由は、まず権限境界が見えていないと、syscall や process の役割分担がぼやけるからです。exec だけを見ても、その先で戻る先が user mode だと分からなければ意味が薄くなりますし、context switch だけを見ても、どの trap frame を保存しているのかが曖昧になります。

この章の到達点
本書を読み終えるころには、少なくとも次を説明できる状態を目指します。

- user mode と kernel mode を分ける理由
- syscall entry で何を保存し、どこで dispatch するか
- `exec` が新しい address space と初期 user stack をどう用意するか
- `fork` と context switch の違い
- `wait` と zombie がなぜ必要なのか

なぜ今これを学ぶのか

`rust-os-book` の流れで見ると、ページングとヒープまで来た時点で「次は user process を動かしたい」と自然に思います。しかしそのためには、単に新しい address space を作るだけでは足りません。特権遷移、trap frame、kernel stack、syscall entry、scheduler の都合まで一気に増えます。本書は、その増えた層を順番にほどくためのものです。

`elf-linker-loader-primer` との接続も強いです。あちらは `execve` の前後で何がロードされるかを扱いましたが、本書は「ロード後に kernel がどのように user mode へ制御を渡すか」を扱います。つまり、`_start` へ飛ぶ直前の話です。

さらに `go-runtime-primer` とも接続します。Go runtime では syscall block や `netpoll` が重要でしたが、それを深く理解するには OS 側の syscall と process の直感があるとかなり楽になります。user-space runtime が何を肩代わりし、何を kernel に依存しているかが見えるからです。

対象と範囲

対象は x86-64 固定です。特権レベル、trap frame、`iretq`、`syscall/sysret`、page table 切り替えといった具体例を揃えやすく、既存の `rust-os-book` と `elf-linker-loader-primer` とも自然につながるからです。

一方で、初版では次を主題にしません。

- file system や pipe
- signal と async I/O
- `epoll` や `kqueue`
- SMP 上の高度な scheduler
- Linux kernel source の完全読解

初版の目標は、`fork/exec/wait` までで process lifecycle を閉じることです。I/O は面白いですが、本筋に入れると process 本体の説明が薄くなります。

x86-64 の説明方針

本書では、x86-64 の具体性を保ちながらも、説明は「教育用の最小 kernel モデル」に寄せます。つまり、最初に trap / interrupt 的な枠で user から kernel へ入る流れを理解し、そのうえで実際の x86-64 では `syscall/sysret` の専用経路がある、と見る方針です。

こうする理由は、`rust-os-book` で既に例外と割り込みを見ているからです。既存の感覚を活かしつつ、専用 syscall fast path が何を省略し、何を前提にしているかを見たほうが理解しやすくなります。

> **注意**
> 本書の toy kernel モデルは説明用です。実在 OS の細部、たとえば Linux の全 ABI や security mitigation の詳細をそのまま再現するわけではありません。ここで重視するのは、「どの情報がどの stack にあり、どのタイミングで address space が切り替わるか」です。

参照する一次資料

本書では、次の種類の資料を主に参照します。

- x86-64 の特権遷移と例外・syscall の説明
- `rust-os-book` とその周辺 crate / OSS
- `execve` と process startup の公開資料
- `go-runtime-primer` が触れている syscall / netpoll の接点資料
- 教育 OS としての xv6 など

source を直接読むときも、最初に持つべき問いは同じです。「今保存しているのは user の状態か kernel の状態か」「この切り替えは trap 由来か scheduler 由来か」です。本書はその問いを揃えることを目指します。

> **編集注記**
> process 管理は、一見すると無数の構造体と状態遷移に見えます。しかし骨格は比較的単純です。user mode を安全に走らせ、必要なときだけ kernel へ入り、必要なら別の process へ切り替える。それを実現するために trap frame、kernel stack、page table、process table が出てきます。

各章の見取り図

本書は次の順で進みます。

1. 特権レベル、trap、user mode への移行を整理する
2. syscall path を entry / dispatch / return に分けて理解する
3. process image、address space、`exec` を扱う
4. `fork`、context switch、`wait` まで process lifecycle を閉じる
5. 付録で演習、用語、次に読む source を整理する

この順序には理由があります。`fork` は魅力的ですが、その前に「戻る先の user mode」を理解していないと、process 切り替えの意味が薄くなります。したがって本書では、まず境界そのものから始めます。

rings、trap、user mode

process を作る前に、まず CPU がどのように権限境界を持っているかを整理する必要があります。もし user code が kernel と同じ権限で動くなら、syscall という概念自体が不要です。逆に user mode があるからこそ、「普通の命令ではできないことを kernel に頼む」という形が必要になります。本章では x86-64 の privilege level、trap、user mode への移行を扱います。

この章の到達点

この章の到達点は次の通りです。

- user mode / kernel mode を分ける理由を説明できる
- trap と interrupt が kernel entry の一般形だと分かる
- trap frame が何を保存するのかを説明できる
- `iretq` が user mode へ戻るための出口だと理解する

なぜ権限境界が必要なのか

user program は信用しません。少なくとも OS 設計では、その前提が出発点になります。もし任意の program が page table を書き換えたり、デバイスレジスタを触ったり、他 process の memory を直接読めたりしたら、保護も隔離も成立しません。したがって CPU は、実行主体ごとに許される操作を分ける必要があります。

x86-64 では ring 0 と ring 3 が主役です。ざっくり言えば、ring 0 が kernel、ring 3 が user です。この区別があるから、普通の user code は privileged instruction を実行できず、kernel へ入るには決められた入口を通る必要があります。

page table 上の user / kernel 境界

権限境界は CPU mode だけでなく、page table にも現れます。x86-64 の page table entry には user/supervisor bit があり、user mode から触れてよい page と kernel 専用 page を分けられます。つまり「どの命令が privileged か」だけでなく、「どの address range が見えるか」も OS が決めます。

この観点を持つと、process は単なる register 集合ではなく、独自の address space を持つ実行主体だと見えてきます。後の `exec` や `fork` で address space が主役になる理由もここにあります。

trap は kernel entry の一般形

user code が kernel へ入る経路はひとつではありません。例外、ハードウェア割り込み、そして syscall がいずれも制御移動を起こします。しかし CPU 側から見れば、いずれも「今の実行文脈を保存し、特権側の handler へ飛ぶ」経路です。この共通枠が trap です。

ここで trap frame という考え方が便利になります。user 側の `rip`, `rsp`, `rflags`、必要な general-purpose register などをまとめて保存しておけば、後で「どこへ戻るか」を再構成できます。つまり trap frame は、user 文脈のスナップショットです。

kernel stack が必要な理由

user stack は user process 自身の所有物です。そこへ kernel の内部処理を積み重ねるのは危険です。user memory が壊れているかもしれませんし、権限分離の意味も薄れます。そこで process ごとに kernel stack を持ち、kernel entry 後の処理はそちらで進めるのが自然です。

この設計は後の context switch でも効きます。process ごとに kernel stack と trap frame を持っていれば、どの process がどの状態で kernel に入っていたかを整理しやすくなるからです。

user mode へ「降りる」とは何か

kernel が user process を初めて走らせるとき、やるべきことは単純ではありません。単に `jmp` で user entry へ飛ぶだけでは足りません。user 用 page table、user stack pointer、user `rip`、適切な `rflags`、segment まわりの前提を揃えたうえで、特権レベルを下げて戻る必要があります。

教育用として分かりやすいのは、「trap から戻る形を人工的に作る」と考えることです。つまり kernel が user 用 trap frame を事前に作り、それを `iretq` で消費して user mode へ戻すイメージです。これで「初回起動」と「通常の trap return」を同じ枠で説明しやすくなります。

`iretq` と `syscall/sysret`

x86-64 には一般的な trap return として `iretq` があり、専用の syscall fast path として `syscall/sysret` があります。本書ではまず `iretq` 的な一般像で理解し、そのあとで「実際の syscall はもっと専用の経路を使うことがある」と見る方針を取ります。

この順序には意味があります。`syscall/sysret` は速いですが、前提も専用です。最初に専用経路だけを見せると、「なぜ user 文脈を別 stack に退避するのか」「なぜ trap frame という考え方が必要か」が見えにくくなります。

TSS と stack 切り替え

x86-64 では、特権遷移時の stack 切り替えや例外時の特別 stack に TSS が関わります。`rust-os-book` では主に例外と IST の文脈で見ましたが、user mode を導入すると「ring 3 から ring 0 に入るときの kernel stack」をどこから取るかが重要になります。

つまり TSS は古い segmentation の名残として覚えるより、「特権遷移の足場を渡すための構造」と見るほうが実用的です。

> **注意**
> user mode を導入するとき、最初に壊れやすいのは user `rip` そのものより stack と page table の整合です。user stack がまだ map されていない、kernel stack を切り替える前に push してしまう、といった順序ミスが典型です。

この章の出口

ここまでで、「なぜ syscall が必要か」「kernel entry は何を保存するか」「どうやって user mode へ戻るか」という土台はできました。次章ではその上に、実際の syscall path を entry、dispatch、return に分けて載せます。

syscall path

権限境界が見えたら、次は user program が kernel に仕事を頼む入口を見ます。本章の主題は syscall です。特別な命令や trap を使って kernel へ入り、引数を受け取り、kernel 側 handler を dispatch し、結果を user に返すまでを一本の流れとして整理します。

> **この章の到達点**
> この章の到達点は次の通りです。

- syscall の entry / dispatch / return を分けて説明できる
- user ABI と kernel 内部表現をどう橋渡しするか分かる
- `copyin/copyout` が必要な理由を説明できる
- trap return と syscall return の共通点と違いを説明できる

syscall は「特別な関数呼び出し」ではない

見た目は関数呼び出しに似ていますが、syscall は普通の call ではありません。なぜなら、呼び出し先が別 privilege level にあり、address space 保護の境界を越えるからです。user 側は number と引数を決められた register へ載せ、専用の入口命令を叩きます。kernel 側はそれを受け取って内部 handler へ振り分けます。

この時点で重要なのは、user ABI と kernel 内部 API は同じでなくてよいことです。user 側では register と integer で渡しても、kernel 側では trap frame や syscall context として一度まとめ直したほうが扱いやすいです。

entry で何を保存するか

syscall entry 直後の最優先事項は、user 文脈を壊さず保存することです。最低限必要なのは次です。

- user `rip`
- user `rsp`
- flags
- syscall number
- 引数 register

実際の x86-64 Linux では、専用の syscall ABI と machine-specific な保存規約があります。本書の toy kernel では、それを trap frame にまとめて「今どの process が、どの引数で kernel に入ったか」を表現できれば十分です。

dispatch table

entry が済んだら、次は syscall number を見て handler へ振り分けます。最小モデルでは、固定長の dispatch table が分かりやすいです。

```
static sys_fn table[] = {
  [SYS_write] = sys_write,
  [SYS_exit]  = sys_exit,
  [SYS_fork]  = sys_fork,
  [SYS_exec]  = sys_exec,
  [SYS_wait]  = sys_wait,
};
```

この形を取ると、syscall は「user が直接 kernel 内関数を呼ぶ」のではなく、「番号で表された要求を kernel が解釈して処理する」ことが見えます。これは security 的にも実装整理としても重要です。

`copyin/copyout` はなぜ必要か

syscall の引数が integer だけなら簡単ですが、多くの syscall は user buffer や文字列 pointer を受け取ります。ここで kernel は、user が渡した pointer をそのまま無条件には信用できません。まだ map されていないかもしれませんし、kernel 領域を指しているかもしれませんし、途中で page fault するかもしれません。

そこで `copyin/copyout` が必要になります。つまり「user pointer を検証し、必要なら fault を扱いながら kernel 安全領域へコピーする」処理です。これがあるから、syscall 境界は単なる関数呼び出しではなく、保護境界として意味を持ちます。

返り値と失敗

syscall は常に成功するとは限りません。最小モデルでも、少なくとも次の失敗を扱います。

- 未知の syscall number
- user pointer 検証失敗
- resource 不足
- 存在しない child への `wait`

返り値は通常 register に載せて user へ返しますが、重要なのは「failure を表す一貫した規約」を持つことです。Linux では `-errno` 系の規約がありますが、toy kernel では負値や明示的エラーコードでも構いません。大事なのは境界で一貫していることです。

syscall return

handler が終わったら、kernel は user 文脈へ戻ります。このとき、「同じ process へすぐ戻る」のか、「scheduler を挟んで別 process へ行く」のかはまだ確定していません。つまり syscall return は、単なる epilogue ではなく scheduler と接続する分岐点です。

この視点はかなり重要です。syscall から戻る経路と timer interrupt から戻る経路は、最後に user 文脈へ戻るという意味では似ています。違いは、その前にどこまで process 管理の判断を挟むかです。

`write` と `exit` から始める理由

最初の syscall としては `write` と `exit` が教材向きです。

- `write` user buffer を kernel へ安全に渡す必要がある
- `exit` process lifecycle に直結する

この二つだけでも、copyin、返り値、process 終了、親側回収の必要性がかなり見えます。本書ではそこへ `fork`, `exec`, `wait` を足して process 本体へ進みます。

> **注意**
> syscall 実装の最初の失敗は、handler 本体より user pointer 検証の抜けです。user buffer を kernel で直接 dereference してしまうと、保護境界の意味が崩れます。`copyin/copyout` を syscall path の一部として最初から考えるのが安全です。

この章の出口

ここまでで syscall は一本の path として見えました。しかし process 本体はまだ曖昧です。次章では、trap frame と kernel stack を持つ実行主体として process を定義し、`exec` で新しい image をどう作るかを見ます。

process と address space

syscall path が見えたら、次はその呼び出し元である process 自体を整理します。process は単なる PID ではありません。address space、kernel stack、trap frame、状態、親子関係などをまとめて持つ実行主体です。本章では、process image と `exec` を主線に、address space の切り替えと user stack 構築を扱います。

この章の到達点
この章の到達点は次の通りです。

- process が持つ最小状態を列挙できる
- address space 切り替えが process 管理の中心だと分かる
- exec が「新しい process を作る」のではなく「既存 process の image を差し替える」と説明できる
- 初期 user stack に何を置くかを説明できる

process は何を持つか

最小モデルでは、process は少なくとも次を持ちます。

- PID
- 実行状態
- page table / address space
- kernel stack
- trap frame
- 親子関係
- 終了コード

ここで trap frame が process 構造体の一部として出てくるのは自然です。なぜなら process が kernel へ入ったときの user 側文脈を表しているからです。scheduler が後でその process を再開したいなら、trap frame をどこかに保持しなければなりません。

address space が process を分ける

複数 process を持つとは、単に複数の `rip` を持つことではありません。どの仮想アドレスが何へ写っているか、つまり page table まで含めて process ごとに持つことです。これがあるから、同じ 0x400000 という user address でも、別 process では別の物理ページを指せます。

この意味で process 管理の中心は address space 管理です。`fork` も `exec` も、結局はどの page table を使い、どの user memory を見せるかの話に戻ります。

user stack をどう作るか

`elf-linker-loader-primer` で見たように、program startup には `argc`, `argv`, `envp`, `auxv` などが初期 stack に載ります。toy kernel ではそこまで全部を再現しなくても構いませんが、「user mode に渡す最初の stack が空ではない」という点は重要です。

最小モデルでも次の発想は持ちたいです。

1. user stack 領域を map する
2. 引数文字列や `argv` 配列を user memory へ置く
3. user `rsp` をその先頭へ向ける
4. user `rip` を entry point へ向ける

これがあるから exec は単なる page table 差し替えではなく、「新しい image を起動可能な形に整える」処理になります。

exec は何をするか

exec は新しい process を作りません。既存 process の実行イメージを差し替えます。この一点は極めて重要です。PID は通常そのままで、親子関係も引き継ぎます。しかし user memory、user stack、entry point、program image は新しくなります。

この操作を最小化すると、exec の仕事は次のようになります。

1. 新しい executable を開いて読み込む
2. 新しい user address space を作る
3. segment を map し、必要なら page を埋める
4. 初期 user stack を作る
5. trap frame の user `rip/rsp` を新 image に向け直す

つまり exec は、「次に user mode へ戻ったときの世界」を丸ごと差し替える操作です。

kernel stack と trap frame の置き場所

process の user image を差し替えても、kernel stack と process 管理構造そのものは kernel 側に残ります。この分離が大事です。kernel は user memory を捨てても、自分がその process を管理するための足場は捨てません。だから exec 後も同じ PID の process として扱えます。

この視点を持つと、「user memory は process の一部だが process そのものではない」と分かります。process は user image より一段広い概念です。

scheduler と process 状態

この段階で最低限持っておきたい process 状態は次です。

- runnable
- running
- blocked / sleeping
- zombie

本格的な scheduler を作らなくても、この 4 つを持つだけで `wait` や context switch をかなり説明しやすくなります。`exit` 後すぐに process 構造体を消せない理由も、zombie を入れると自然に見えます。

> **注意**
> exec を「新しい process を作る syscall」だと覚えると、`fork` との役割分担が崩れます。`fork` は実行主体を複製し、exec は image を入れ替える。この分離が Unix 系 process モデルの核心です。

この章の出口

ここまでで process image と address space は見えました。次章ではそれを複製し、切り替え、終了を回収する話へ進みます。`fork`、context switch、`wait` がそこで一つにつながります。

`fork`、exec、`wait`、context switch

process と address space が見えたら、最後に process lifecycle を閉じます。本章では `fork`、exec、`wait` を主線に、context switch と scheduler の最小像を整理します。目標は、「どの瞬間に process が増え、どの瞬間に image が入れ替わり、どの瞬間に実行主体が切り替わるか」を混同せず説明できるようになることです。

この章の到達点
この章の到達点は次の通りです。

- `fork` と `exec` の役割分担を説明できる
- context switch と trap return の違いを説明できる
- `wait` と zombie が必要な理由を説明できる
- process lifecycle を syscall と scheduler の両方から見られる

`fork` は何を複製するか

`fork` の最小意味は、「今の process によく似た新しい process を作る」です。最も素朴な実装では、親の address space を丸ごと複製し、trap frame もほぼコピーします。違いは戻り値だけです。親には child PID、子には 0 を返します。

この設計が Unix っぽいのは、`fork` が「新しい image を指定しない」点にあります。いったん今の実行状態を複製し、その後に必要なら child 側で `exec` を呼んで別 image へ移ります。この二段階が process 制御を柔軟にしています。

copy-on-write は改善策であって本質ではない

production OS は `fork` のたびに全 address space を丸ごとコピーしません。copy-on-write を使い、実際に書き込みが起きるまで page を共有します。しかし教材として最初に大切なのは最適化ではなく、「`fork` は execution context を複製する」という意味です。copy-on-write はその高価さを和らげる改善策です。

したがって本書では、本質を次のように押さえます。

- 親と子は最初ほぼ同じ user image を持つ
- 戻り値だけが違う
- scheduler は以後それぞれを独立 process として扱う

exec は process を消さず image を替える

`fork` で child を作ったあと、典型的には child が `exec` を呼んで別プログラムへ入れ替わります。ここで重要なのは、`exec` は PID を変えないことです。つまり、親から見ると「同じ child process が別 program になった」だけです。

この `fork` + `exec` の分業があるから、shell は child を作ってからその中身だけ差し替えられます。もし `exec` が新 process 生成まで兼ねていたら、親子関係や待ち合わせの構図はかなり変わってしまいます。

context switch は何を切り替えるか

context switch は「今 CPU に乗っている kernel 実行文脈を別 process の文脈へ切り替える」処理です。ここで混同しやすいのは、syscall return との違いです。

- trap return: 同じ process の user 文脈へ戻る
- context switch: どの process を次に走らせるかを変える

つまり context switch は scheduler 判断を伴います。保存するのは current の kernel 側状態、復元するのは next の kernel 側状態で、その先に trap frame を経由して user mode へ戻るかどうかが決まります。

scheduler の最小像

本書で必要な scheduler 像は最小限で十分です。次の構造があれば process lifecycle を説明できます。

- runnable queue
- current process
- sleep / wake の仕組み
- timer interrupt などの再スケジュール契機

この最小像があると、`wait` で親が sleep し、child の `exit` で wake される流れが自然に見えます。`go-runtime-primer` の scheduler と比べると、こちらは kernel が process を直接切り替える世界だと分かります。

`exit` と zombie

process が終了しても、その場で process 構造体を完全に消すわけにはいきません。親が終了コードを回収する前に全部消すと、`wait` が何も知れなくなるからです。そこで process はいったん zombie 状態になります。これは「実行は終わったが、親への報告がまだ残っている」状態です。

この設計は一見地味ですが、Unix process モデルのかなり本質です。親子関係があるからこそ、終了情報の受け渡しも process 管理の一部になります。

`wait` は何を待つか

`wait` は child の終了を待ち、その終了コードを回収します。child がまだ走っていれば親は sleep し、child が zombie になったら回収して返ります。ここで scheduler と process table が再び接続します。単なる polling ではなく、「親を寝かせ、子の終了で起こす」という同期だからです。

この構造を持つと、`wait` は単なる便利 API ではなく、process lifecycle を閉じるための仕組みだと分かります。

> **編集注記**
> Linux の実際の process 管理は、signal、thread group、namespace、fd table 共有など、ここで扱わない要素が大量にあります。本書の役割は、それらの前にある最小骨格を見せることです。`fork/exec/wait` と context switch の関係が見えれば、巨大な source tree でも入り口を失いにくくなります。

> **注意**
> `fork`、context switch、syscall return を全部「戻る処理」とひとまとめに覚えると混乱します。`fork` は実行主体の複製、context switch は current/next の入れ替え、trap return は user mode への出口です。似た register 保存をしていても、意味は別です。

この章の出口

ここまでで user/kernel 境界の最小 process 世界は閉じました。user mode へ降りる、syscall で入る、`fork` で増える、`exec` で入れ替わる、`wait` で回収する。この一連が見えると、OS 側の process 管理と user-space runtime の責務分担もかなり分かりやすくなります。

付録と次の一歩

本編では、特権境界から始めて、syscall path、process image、`fork/exec/wait`、context switch までを一本につなぎました。この章では、復習、演習、つまずき、用語集、次に読む source をまとめます。process 管理は枝が多いですが、最初は「どの mode で走っているか」「どの stack を使っているか」「どの address space を見ているか」の 3 つへ戻ると整理しやすいです。

章ごとの復習ポイント

本編の核を短く戻すと、各章は次のように整理できます。

- 導入: 今の空白は `execve` の先と user/kernel 境界である
- user mode: trap frame と kernel stack が user 文脈を安全に持ち運ぶ
- syscall: entry、dispatch、`copyin/copyout`、return で構成される
- process: address space、kernel stack、trap frame を束ねた実行主体
- lifecycle: `fork` は複製、`exec` は image 差し替え、`wait` は終了回収、context switch は current/next の切り替え

小さな演習課題

理解を固めるなら、次の順で短い演習が効きます。

1. user mode に入るための最低限の trap frame を紙上で書き出す
2. `write` syscall の引数検証で、どこで `copyin` が必要かを列挙する
3. `exec` 時に新しい user stack へ何を置くかを図にする
4. `fork` 後に親と子で何が同じで何が違うかを表にする
5. `wait` で zombie が必要になる理由を、親子 2 process の時系列で説明する
6. `go-runtime-primer` の syscall block と、本書の kernel scheduler を比較する

大きな実装である必要はありません。短い状態遷移図と register / stack 図のほうが理解に効くことも多いです。

この章の到達点

本書の理解確認として、最低限次を説明できるかを試してください。

- なぜ user stack と kernel stack を分けるのか
- syscall が普通の関数呼び出しではない理由
- `exec` が新 process 生成ではない理由
- context switch と trap return の違い
- `wait` と zombie の関係

よくあるつまずき

exec が child を作る syscall だと思ってしまう

違います。child を作るのは `fork`、image を差し替えるのが `exec` です。

syscall return と context switch を同じものだと思ってしまう

前者は同じ process の user 文脈へ戻る出口、後者はどの process を次に走らせるかの切り替えです。

user pointer を kernel でそのまま触ってしまう

`copyin/copyout` が必要です。ここを抜くと保護境界の意味が壊れます。

zombie を「ただのリーク」と思ってしまう

zombie は親が終了状態を回収するまで残る設計上の待機状態です。不要な残骸ではなく、`wait` のための情報保持です。

trap frame を「割り込み専用構造」と思ってしまう

user/kernel 境界で文脈を保存する枠として見ると、syscall、例外、初回 user entry までまとめて理解しやすくなります。

用語小事典

- user mode 制限付き権限で動く通常の program の実行モード
- kernel mode privileged operation を実行できるモード
- trap 例外、割り込み、syscall を含む kernel entry の一般形
- trap frame user 文脈の保存結果
- kernel stack kernel で処理するときの stack
- address space 仮想アドレスから物理ページへの対応全体
- syscall user が kernel に仕事を依頼する入口
- `copyin/copyout` user memory と kernel memory の安全なコピー
- `fork` process を複製する syscall
- `exec` process の実行 image を差し替える syscall
- `wait` child の終了を回収する syscall
- zombie 実行は終わったが、親の回収待ちで情報が残っている process 状態
- context switch CPU 上の current process を別 process へ切り替える処理

次に読むなら

本書の次に進む方向は大きく 3 つあります。

- OS 側を深める: `rust-os-book` の続きとして user memory 保護、copy-on-write、file descriptor、file system へ進む
- ELF / startup 側を深める: `elf-linker-loader-primer` と往復し、`execve` の前後や user stack 構築をより詳しく追う
- runtime 側を深める: `go-runtime-primer` へ戻り、syscall block、`netpoll`、user-space scheduler との役割分担を見る

参考資料

主な足場は次です。

- `rust-os-book` の割り込み・ページング・次に伸ばす方向
- `elf-linker-loader-primer` の `execve` / startup 部分
- `go-runtime-primer` の syscall / scheduler 接点
- x86-64 の特権遷移資料
- xv6 などの教育 OS

編集注記

process 管理を読むときに大切なのは、構造体名より保存点を見抜くことです。今保存しているのが user 側の状態か kernel 側の状態か、戻り先が同じ process か別 process か。その 2 問が分かれば、長い source でもかなり読みやすくなります。

おわりに

user/kernel 境界は、低レイヤ学習の中でも特に「複数の本が合流する場所」です。ELF と loader はここへ process を渡し、OS はここで保護と切り替えを行い、runtime はこの境界の上で scheduler や GC を組み立てます。

本書が目指したのは、syscall と process を別々の箱に入れず、一つの lifecycle として見られるようにすることでした。ここまで読めたなら、OS 側の実装を伸ばすときも、runtime が syscall とどう付き合うかを見るときも、かなり地図を持って進めるはずです。